18 AUTHENTICATION vs AUTHORIZATION

These two words get used almost interchangeably in casual conversation, but they answer two completely different questions, happen in a fixed order, and — this is the part that trips people up — fail with two DIFFERENT HTTP status codes.

```
AUTHENTICATION   "WHO are you?"          -> verifying identity
AUTHORIZATION    "WHAT are you allowed   -> checking permissions,
                  to do?"                   given a known identity

Client Request
     |
AUTHENTICATION  <-- must know WHO is calling before anything else
     |              can meaningfully decide what they're allowed to do
AUTHORIZATION   <-- now checks: is THIS identified user allowed
     |              to do THIS specific action?
Route Handler runs
```

THE ORDER MATTERS — AUTHORIZATION NEEDS AUTHENTICATION FIRST

You cannot answer "is this user allowed to delete this task?" without first knowing WHO the user is. That's why authentication always runs before authorization in the request pipeline (both are implemented as Guards in Nest — see theory/17-guards.txt — and Nest runs guards in the order they're applied).

```
  @UseGuards(AuthGuard, RolesGuard)     // AuthGuard runs BEFORE RolesGuard
  @Roles('admin')
  @Delete(':id')
  remove(...) { ... }

AuthGuard    verifies the request's credentials (e.g. a JWT), and if
             valid, ATTACHES the identified user to the request:
                 request.user = { id: 7, role: 'admin' }
             If the credentials are missing/invalid, it throws and the
             request stops here — RolesGuard never even runs.

RolesGuard   reads request.user (set by AuthGuard) and the route's
             @Roles('admin') metadata, and decides: does THIS user's
             role satisfy THIS route's requirement?
```

THE STATUS CODE DIFFERENCE (COMMONLY GOTTEN WRONG)

401 Unauthorized	Actually means "you are NOT AUTHENTICATED" — no valid credentials were presented at all, or they were invalid/expired. Despite the name, this is an AUTHENTICATION failure, not an authorization one. ("Unauthorized" is a historically confusing name — think of it as "who ARE you? I can't tell.")
403 Forbidden	"You ARE authenticated — I know exactly who you are — but you're not ALLOWED to do this specific thing." This is a genuine AUTHORIZATION failure.

No token at all	→ 401 (authentication never happened)

Expired/invalid token	→ 401 (authentication failed)
Valid token, wrong role	→ 403 (authenticated fine, not permitted)

TIED TO THIS PROJECT'S theory/17-guards.txt EXAMPLE

The ApiKeyGuard example in theory/17-guards.txt is worth re-examining with this distinction in mind:

```
if (key !== VALID_API_KEY) {
  throw new UnauthorizedException('Invalid or missing API key');
}
```

This is AUTHENTICATION, and a fairly crude form of it — it proves the caller possesses a shared secret (like a service-to-service API key), but it does NOT identify WHICH user is calling, so there's no way to build authorization rules on top of it (no "is this specific person an admin?" — there's no concept of "this specific person" at all). It's fine for a single trusted client (e.g. a backend calling another backend), but not for a multi-user app where different users need different permissions.

A MORE COMPLETE PAIR — JWT AUTHENTICATION + ROLE AUTHORIZATION

```
// auth.guard.ts — AUTHENTICATION: verifies WHO
@Injectable()
export class AuthGuard implements CanActivate {
  constructor(private readonly jwtService: JwtService) {}

  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest();
    const token = request.headers.authorization?.split(' ')[1]; // "Bearer xxx"

    if (!token) {
      throw new UnauthorizedException('No token provided'); // 401
    }

    try {
      request.user = this.jwtService.verify(token); // e.g. { id: 7, role: 'admin' }
      return true;
    } catch {
      throw new UnauthorizedException('Invalid or expired token'); // 401
    }
  }
}

// roles.guard.ts — AUTHORIZATION: checks WHAT they can do
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.get<string[]>('roles', context.getHandler());
    if (!requiredRoles) return true;

    const { user } = context.switchToHttp().getRequest(); // set by AuthGuard
    if (!requiredRoles.includes(user.role)) {
      throw new ForbiddenException('Insufficient permissions'); // 403
```

```
    }
    return true;
  }
}

// task.controller.ts — usage
@Roles('admin')
@UseGuards(AuthGuard, RolesGuard)
@Delete(':id')
remove(@Param('id', ParsePositiveIntPipe) id: number): Task {
  return this.taskService.remove(id);
}
```

Three distinct outcomes for DELETE /task/1 now:

- No Authorization header	→ AuthGuard throws → 401
- Bad/expired token	→ AuthGuard throws → 401
- Valid token, role != 'admin'	→ RolesGuard throws → 403
- Valid token, role == 'admin'	→ both pass → handler runs

SIDE-BY-SIDE TABLE

```
|                     | Authentication              | Authorization                |
|---------------------|-----------------------------|------------------------------|
| Question answered    | Who are you?                | What are you allowed to do?  |
| Runs                 | First                       | After authentication         |
| Failure status code  | 401 Unauthorized            | 403 Forbidden                |
| Typical mechanism    | Password, API key, JWT,     | Roles, permissions, ACLs,    |
|                      | OAuth token, session cookie | ownership checks             |
| In Nest, usually a   | Guard that populates        | Guard (often reading         |
|                      | request.user                | request.user + route metadata)|
| Example question     | "Is this a valid, logged-in | "Is this logged-in user an   |
|                      | user?"                      | admin, or do they own this   |
|                      |                             | specific task?"              |
```

★ RULE OF THUMB

If the check is "does this request prove who's making it" — that's authentication, fails with 401. If the check is "given who they provably are, are they permitted to do THIS" — that's authorization, fails with 403. Never conflate the two status codes: returning 401 for a permission problem (or 403 for a missing/bad token) misleads API clients about what actually went wrong and how to fix it.

See also: theory/17-guards.txt, theory/07-dependency-injection.txt, theory/06-decorators.txt